

**ΟΙΚΟΝΟΜΙΚΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΑΘΗΝΩΝ**



ATHENS UNIVERSITY
OF ECONOMICS
AND BUSINESS

Hyperparameter Optimization of a Scheduling Problem via Surrogate-Assisted Search

Author: Panagiotis Theocharis

Supervisor: Prof. Ioannis Mourtos

Contributor: Stavros Vatikiotis

Department of Management Science and Technology

July 2025



AutoML Meets Metaheuristics

Hyperparameter Optimization of a Scheduling Problem via
Surrogate-Assisted Search

Author: Panagiotis Theocharis

Supervisor: Prof. Ioannis Mourtos

Contributor: Stavros Vatikiotis

Department of Management Science and Technology

July 2025

Contents

Abstract	3
1 Introduction	5
1.1 Background	5
1.2 Research Objectives	6
1.3 Contribution	7
1.4 Structure of the Thesis	8
2 Theoretical Foundations	10
2.1 Scheduling Problems	10
2.1.1 Unrelated Parallel Machine Scheduling (α)	11
2.1.2 Sequence-dependent and machine-dependent setup times (β)	11
2.1.3 Weighted Tardiness (γ)	11
2.1.4 Genetic Algorithm	12
2.2 Hyperparameter Optimization	13
2.2.1 Traditional HPO Methods	13
2.3 Bayesian Optimization	13
2.3.1 Gaussian Process (GP) Regression	14
2.3.2 Expected Improvement (EI)	16
2.3.3 Tree Parzen Estimator (TPE)	17
3 Literature Review	19
3.1 Introduction to the Literature Landscape	19
3.2 Manual Tuning and the Motivation of Automation	20
3.3 Capabilities and Impediments of HPO	21
3.4 Applications of HPO and AutoML in Optimization and Metaheuristics	23

4	Experimental Design	25
4.1	Genetic Algorithm Implementation and Adaptation	25
4.2	Dataset	26
4.3	AutoML Wrapper and Optimization Pipeline	27
4.4	HPO Strategy Configuration	30
4.4.1	Search Space Configuration	30
4.4.2	Additional Configurations	31
4.5	Selected Metrics	33
4.6	Assumptions	35
4.7	Limitations	36
5	Results	38
5.1	Solution Quality and Variability	38
5.1.1	Solution Quality by Problem Size	38
5.1.2	Results by α, τ	42
5.2	Computational Cost	47
5.2.1	Execution Time and Efficiency Analysis	47
5.3	Configurations Tested Deltas	50
5.4	Discussion	51
5.4.1	General Observations	51
5.4.2	Uncovering the Best-Case Scenarios	52
6	Conclusion	54
6.1	Summary of Findings	54
6.2	Future Work	55
6.3	Closing Remarks	56

Abstract

This thesis investigates the potential of Automated Machine Learning (AutoML) techniques in tuning the hyperparameters of a metaheuristic algorithm applied to a complex scheduling problem. Traditional metaheuristic methods of optimization often rely on manual parameter tuning, a process that is time-consuming, inefficient, and depends on expert intuition. AutoML, originally developed to streamline the deployment of machine learning (ML) models, automates key elements of the ML pipeline, most notably, Hyperparameter Optimization (HPO). This work explores the potential of applying AutoML HPO strategies beyond ML, using them to tune a Genetic Algorithm for a representative operations research problem. The accessibility of advanced HPO techniques through AutoML frameworks, usable without extensive ML expertise, presents a compelling opportunity to enhance performance in complex optimization tasks.

To this end, the study focuses on minimizing weighted tardiness in unrelated parallel machines with sequence-dependent setup times, a problem previously addressed using a Genetic Algorithm (GA) in the article “Weighted tardiness minimization for unrelated machines with sequence-dependent and resource-constrained setups” by Avgerinos et al. With permission, their GA implementation was used as the foundational resource for experimentation. To reduce computational complexity and isolate HPO analysis, the constraint programming component was removed, allowing the study to focus specifically on evaluating AutoML’s effect on GA performance.

The tuning process was approached by integrating AutoML frameworks, treating the GA as a black-box function, and benchmarking results from AutoML, and manually tuned versions. Classic methods were not the focus of the core experimentation due to their computational burden in this already demanding problem setting. Instead, two more efficient techniques, specifically a Gaussian Process surrogate with an Expected Improvement acquisition and Tree-Parzen Estimators were employed. The Grid Search from the manual tuning was included only as a baseline in the final benchmarking to

contrast with the performance of the more sophisticated methods.

Chapter 1 Introduction

1.1 Background

In recent years, Machine Learning (ML) has gained significant traction, leading to widespread adoption in both industry and academia. A 2023 bibliometric analysis by Ezugwu et al. (16) revealed a notable upsurge in highly cited ML publications over the past decades. Furthermore, the Journal of Machine Learning Research (JMLR), a leading outlet in the field, reported an average of 250 papers annually as of 2022. This surge has broadened the breadth and complexity of ML implementations, driving an increased demand for data scientists. However, developing effective ML systems typically requires highly skilled professionals across various stages of the end-to-end pipeline (50), a requirement that many practitioners may not fully meet.

In response to this skill gap, the development of Automated Machine Learning (AutoML) toolset has emerged. These systems aim to automate fundamental tasks within the ML workflow, thus democratizing access to ML technologies.

Although AutoML was originally intended to address challenges within ML, its utility can extend to adjacent domains, such as operations research and combinatorial optimization. One particularly relevant aspect is Hyperparameter Optimization (HPO), a key step in the ML pipeline for boosting model performance. Automating HPO reduces manual effort, increases efficiency, and enhances reproducibility (29). Similarly, in combinatorial optimization, considerable effort is invested in tuning parameter configurations to improve convergence to optimal solutions. A traditional class of algorithms that rely significantly on such tuning are metaheuristics.

Metaheuristics have been studied since the 1960s and gained traction in the 1980s as extensions of heuristic methods for more complex problems. The advancement's scope was to explore large search spaces efficiently, avoid premature convergence to local optima, and offer alternatives to traditional deterministic or exact methods (23). Often inspired by natural phenomena, such as biological evolution, animal behavior, or

physical processes, metaheuristics have proven significantly effective for addressing NP-hard problems where exact solutions are computationally impractical and costly (9). Notable examples include Genetic Algorithms, inspired by natural selection; Particle Swarm Optimization, based on the social behavior of bird flocks; Ant Colony Optimization, modeled after the foraging patterns of ants; and Simulated Annealing, which emulates the annealing process in metallurgy. Each of these approaches involves numerous hyperparameters, such as the mutation rate in Genetic Algorithm or inertia weight in Particle Swarm Optimization, that critically influence performance and require careful tuning. Given this sensitivity, the role of HPO in metaheuristics establishes AutoML frameworks as promising tools for automating their configuration.

To delve into this opportunity, this thesis focuses on a case study involving a weighted tardiness minimization problem with sequence-dependent setup times in an unrelated parallel machine environment, solved using a Genetic Algorithm (GA). The authors of the article “Weighted tardiness minimization for unrelated machines with sequence-dependent and resource-constrained setups” (2) have granted permission to adapt their GA implementation as an exemplification and apply HPO to an adjacent problem. The difference between the problem formulations is further explained in Sect. 4.1. . Despite this distinction, the problem retains its NP-hard complexity, ensuring the relevance of the experimental evaluation.

1.2 Research Objectives

The primary objective of this research is to investigate the potential of Automated Machine Learning (AutoML) techniques in enhancing the tuning process of metaheuristic algorithms for complex combinatorial optimization problems. As stated before, the study focuses on the Weighted Tardiness Minimization Problem with sequence-dependent setup times in an Unrelated Parallel Machine environment, solved using a GA. Therefore, the specific research objectives are as follows:

- To evaluate the applicability of AutoML frameworks in automating hyperparameter optimization (HPO) for an exemplary metaheuristic.
- To benchmark the optimization outcomes of two different HPO strategies, specifically Bayesian Optimization variants, against traditional methods and the variants themselves.

- To analyze the influence of GA hyperparameters on the solution quality and convergence behavior of the algorithm under different HPO strategies.
- To contribute toward bridging the gap between ML and the traditional optimization domain, by demonstrating how AutoML performs in classical optimization problems.
- To lay the groundwork for future research by identifying key bottlenecks and areas of improvement in the final proposed framework.
- To evaluate the reproducibility and transferability of AutoML tuning within the tested metaheuristic context.
- To demonstrate the potential of AutoML for metaheuristic tuning through a focused case study, without aiming to deliver an exhaustive review or definitive generalizations.

Collectively, these research objectives lay the foundation for identifying the practical capabilities and limitations of AutoML, setting the stage for the contributions of this study.

1.3 Contribution

This thesis contributes to the underexplored intersection of Automated Machine Learning (AutoML) and combinatorial optimization by systematically evaluating the integration of advanced hyperparameter optimization techniques, specifically Bayesian Optimization variants such as Gaussian Process with Expected Improvement (GP+EI) and Tree-structured Parzen Estimators (TPE), into the tuning process of a Genetic Algorithm (GA). The study addresses a representative and relevant problem: minimizing weighted tardiness on unrelated parallel machines with sequence-dependent setup times.

A primary methodological contribution is the design and implementation of a modular AutoML-GA wrapper that encapsulates the metaheuristic algorithm as a black-box objective. This wrapper facilitates experimentation by not being dependent to specific methods, providing interchangeability. It can be imagined as a pocket that can fit any method a user or researcher desires. Additionally, it ensures reproducibility across diverse scheduling instances without requiring specific domain modifications to the algorithm itself.

Empirically, in the tested scheduling problem, AutoML approaches not only matched, but also mostly surpassed manual tuning, particularly in scenarios characterized by increased dimensionality and complexity. In smaller or simpler instances, however, observed gains were modest, highlighting limitations dependent on context. Notably, the GP+EI approach showed enhanced sensitivity and stability under taxing conditions, such as instances with high setup intensity, achieving meaningful median improvements up to 16.7%. TPE, despite exhibiting greater performance volatility, still provided substantial benefits in specific high-dimensional landscapes.

Crucially, these performance enhancements were accompanied by notable computational savings, particularly in larger scale scenarios. Beyond performance, this thesis contributes to the broader effort of bridging operations research and ML by applying AutoML, originally designed for ML model configuration, to classical optimization tasks. Ultimately, the core contribution lies in demonstrating the potential of AutoML optimization as a viable alternative to manual configuration in operations research. This work serves as a foundational step toward future research aimed at refining and tailoring AutoML methodologies to better accommodate the structural complexities and stochastic nature of metaheuristics and other optimization frameworks that necessitate parameterization.

1.4 Structure of the Thesis

The remainder of this thesis is structured as follows.

- **Chapter 2** explains the core theoretical concepts of this work, including the Weighted Tardiness Scheduling Problem, an overview of Genetic Algorithms, and the foundational principles of HPO and Bayesian Optimization. It aims to improve reader comprehension, not to be exhaustive.
- **Chapter 3** presents the relevant literature, discussing previous implementations of manually tuned grid search, previous applications of genetic algorithms, common tuning strategies, and the integrative attempts of AutoML in combinatorial optimization.
- **Chapter 4** details the implementation of the GA and the experimental setup, including the automation of the wrapper, toolsets, metrics, baselines, special statistical treatment, and the configuration of the HPO strategy.

- **Chapter 5** presents and interprets the experimental results, focusing on benchmarking insights, deviation patterns, areas of improvement and identification of the potential.
- **Chapter 6** concludes the work, reflects on objectives, discusses implications, and outlines possible future research paths.

Chapter 2 Theoretical Foundations

This chapter outlines the essential theoretical concepts that underlie the study. It is intended to support comprehension for readers without a specialized background. It begins by defining the core elements of the scheduling problem addressed, namely, Unrelated Parallel Machine Scheduling (UPMS) with sequence-dependent setup times and the objective of minimizing total weighted tardiness. The chapter then introduces Genetic Algorithms (GAs), the sole metaheuristic used in this thesis, emphasizing their structure, tuning needs, and relevance to combinatorial optimization. Finally, it presents the principles of Hyperparameter Optimization (HPO), with a focus on Bayesian Optimization (BO) and its variants, establishing the cornerstone behind the AutoML techniques adopted in this work.

2.1 Scheduling Problems

The concept of scheduling problems refers to the optimal allocation of job time intervals across one or more separate machines, effectively assigning jobs on a set of machines. This allocation involves various restrictions that need to be met, thus increasing the complexity of the problem (11). Each job-machine configuration is subject to an equivalent processing time for task completion.

To systemically describe the details of the problem, Graham’s widely adopted notation ($\alpha|\beta|\gamma$) (25) is used as a formal structure, where α refers to the machine environment, β clarifies job and machine-related constraints and γ denotes the objective. In this thesis, the framing is set as an Unrelated Parallel Machine Scheduling environment, incorporating sequence-dependent and machine-dependent set-up times constraints and the objective of minimizing total weighted tardiness, making it a representative case of this notational structure.

2.1.1 Unrelated Parallel Machine Scheduling (α)

The phrase Unrelated Parallel Machine Scheduling (UPMS) refers to an environment in which multiple machines work in parallel, each having different processing times for the same job without adhering to a predictable pattern, which explains the unrelated adjective. Thus, for a total of n jobs in the set J and a total of m machines in the set M , each job has a different processing time p_{ij} , when placed on each machine i (49). UPMS is the most complex variant in a Parallel Machine Scheduling setting, due to the heterogeneity of processing times significantly increasing problem complexity (47).

2.1.2 Sequence-dependent and machine-dependent setup times (β)

In turn, this constraint belongs to the β section of Graham's notation. Setup times refer to the time required to prepare either a machine or a job prior to the completion of a task. This typically involves the allocation of human resources to enable the setup process. The term sequence-dependent indicates that the setup time varies depending on the job order on a given machine. Similarly, machine-dependent setup times vary according to the specific operation requirements of the operating machine.

Within this previously defined framework (a set of J jobs and a set of M machines), these setup times are denoted as s_{ijk} and are viable to be classified as constraints, since the model must account for their influence during evaluation (47). Further details regarding the reasoning behind the configuration of the problem constraints are discussed in Sect. 4.1.

2.1.3 Weighted Tardiness (γ)

As the main objective of the problem, defining the weighted tardiness adds clarity to the optimization context. It refers to the cumulative penalty incurred from completing jobs past their due dates, where each job has an associated weight that signifies its relative importance. The tardiness of a job is defined as the positive difference between its completion time and due date, and the weighted sum aggregates these penalties across all jobs. This metric allows the prioritization of more critical jobs over others, aligning with practical scheduling goals in manufacturing and service systems (41). The influence of both machine configurations and resource constraints further affects this measure, making it sensitive to the overall environment and the sequencing logic.

2.1.4 Genetic Algorithm

Genetic Algorithms (GAs) are a class of population-based metaheuristics inspired by the principles of natural selection and evolution (39). They operate by maintaining a set of candidate solutions, termed a population, and iteratively evolving them toward higher quality solutions through biologically inspired operators: selection, crossover, and mutation. Each candidate solution is typically encoded as a chromosome, representing a possible solution to the problem at hand. These individuals are evaluated using a fitness function that quantifies their quality. Over successive generations, individuals with higher fitness are more likely to be selected, allowing superior traits to propagate through the population (24).

The algorithm is governed by four primary hyperparameters: population size, number of generations, mutation rate, and crossover rate. These hyperparameters are independent to problem uniqueness. The population size determines the number of candidate chromosomes, while generations define the number of evolutionary cycles. Crossover combines two selected parent chromosomes by mixing their components to produce off-springs after certain modifications, promoting genetic diversity, while mutation introduces random alterations to individual chromosomes, helping to escape local optima. Both crossover and mutation usually encompass random generators that determine if the chromosome will receive the respective modification or not. These probability rates are the ones that must be predetermined before running the algorithm.

In optimization problems, particularly combinatorial ones such as scheduling, GAs are highly valued for their flexibility, sturdiness, and ability to explore complex, non-linear, and multi-modal search spaces without requiring gradient information or convexity assumptions (26) (27).

However, the effectiveness of GAs is significantly influenced by the proper tuning of these hyperparameters. Suboptimal configurations can result in slow convergence, entrapment in local optima, or unnecessary computational overhead (14). Each hyperparameter, if misconfigured, can naturally provoke its own set of issues. For instance, increasing population and generations affects computational overhead directly. This sensitivity to configuration highlights their suitability for integration with Hyperparameter Optimization (HPO) frameworks, which can combat these issues dynamically, a core focus of this thesis.

2.2 Hyperparameter Optimization

Hyperparameters are predefined settings that govern algorithm behavior but are not learned from data. (18). In ML, they include learning rate, tree depth, or batch size; In metaheuristics, they comprise of parameters that usually define the structural layout of the algorithm or perform modification operations, such as population size, mutation rate, and iteration limits (9). These parameters impact the algorithmic behavior directly and as they are predetermined, are not learned from data. These properties clarify that they can be classified as hyperparameters.

Hyperparameter Optimization (HPO) is the task of automating the tuning process to identify the best-performing configuration for a given objective. Studies have shown that customized tuning consistently outperforms default, out-of-the-box settings (37) (40), highlighting the need for flexible processes that adapt to the specific characteristics of each case.

2.2.1 Traditional HPO Methods

Traditional HPO methods, such as Grid Search and Random Search, explore predefined search spaces using exploratory techniques. Grid Search exhaustively evaluates all combinations within a discrete set of values, while Random Search samples configurations randomly within the defined space (5). These approaches are particularly effective when the search space is well defined and the relationships between hyperparameters and performance are complex but not entirely unknown. Grid Search limitations are elaborated upon in sections 3.2 and 3.3, as it enacts as the baseline of comparison in this experiment.

2.3 Bayesian Optimization

A more advanced, yet classical, and widely adopted HPO method is Bayesian Optimization (BO). BO belongs to the class of model-based optimization algorithms and has become particularly valuable in machine learning, where objective functions, such as validation loss, are often expensive to evaluate (44). BO treats the objective function as a black box, meaning it does not require any internal knowledge of the function but relies solely on input-output observations (21). The goal is to identify the input configuration that either minimizes or maximizes the objective function as efficiently as possible.

The process begins with a small initial set of evaluations, followed by iterative updates using a surrogate model, typically a probabilistic model such as a Gaussian Process or a tree-based alternative, which provides a posterior estimate of the objective across the search space. An acquisition function, such as Expected Improvement or Knowledge Gradient, uses this surrogate to assign scores to candidate points by balancing exploration of uncertain regions with exploitation of promising ones (21). The point that maximizes the acquisition function is then selected for the next evaluation. This cycle continues until a predefined stopping criterion is met, such as a maximum number of iterations or a time budget.

BO’s probabilistic and adaptive nature makes it highly sample-efficient, a key advantage when training machine learning models is computationally costly. It is widely used for tasks such as neural network tuning, reinforcement learning optimization, and scientific discovery.

Furthermore, BO is a flexible framework with multiple variants designed for different use cases, including multi-fidelity optimization, constrained BO, and batch evaluations (46). These properties directly address challenges often encountered in metaheuristics, namely, the high cost of evaluations and the reliance on trial-and-error tuning. Since BO is itself a model-based optimizer, the metaheuristic can naturally act as the black-box function being tuned, while BO performs the search over its configuration space. Next, the different BO components that were used in this thesis will be explained in more detail from a theoretical standpoint.

2.3.1 Gaussian Process (GP) Regression

A Gaussian Process (GP) is a non-parametric regression model widely used in BO, particularly within the context of HPO frameworks. GPs define a posterior distribution over functions, providing flexibility in modeling complex, non-linear relationships between inputs and outputs. This makes them particularly suitable for AutoML tasks, where the goal is to optimize an objective function with high-dimensional and non-linear hyperparameters. In a GP framework, the model is specified by two critical components: the mean function $\mu(x)$ and the covariance function (or kernel) $k(x, x')$, both of which are used to capture key properties of the objective function being optimized (42). The kernel function contributes to the formulation of the covariance matrix.

The mean function $\mu(x)$ represents the predicted output for a given input x and is often assumed to be zero unless additional domain knowledge suggests otherwise. In mathematical terms:

$$\mu(x) = \mathbb{E}[f(x)]$$

Where:

- $f(x)$ denotes the objective function
- $\mathbb{E}[f(x)]$ denotes the expected value of the function input x (42).

The **covariance function** $k(x, x')$, also known as the kernel, specifies the relationship between different input points x and x' . It encodes the smoothness and other properties of the underlying function (42).

The **kernel function** is often selected based on prior knowledge or assumptions about the smoothness of the function being modeled. For example, some common choices include the Radial Basis Function (RBF) kernel, also known as the Squared Exponential, the Matérn kernel and the Rational Quadratic kernel. These are widely integrated into AutoML frameworks, and their mathematical formulations are well-established, but omitted here to maintain clarity and improve readability.

The **covariance matrix** $\mathbf{K}$, given a set of observed points $\mathbf{X}$ and their corresponding function values $\mathbf{y}$, is a central component of the GP framework. It is computed using the kernel function applied to all possible pairs of observed inputs $\mathbf{x}_i, \mathbf{x}_j$ resulting in a symmetric square matrix with the following structure:

$$\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$$

The posterior distribution of $f(x^*)$, where x^* is a new input, is central to the predictive capability of GP regression. After conducting a set of observations, the GP provides an expression for the posterior (assumed to be Gaussian). This posterior is characterized by a mean $\mu(x^*)$ and variance $\sigma^2(x^*)$, calculated by using the specified mean and covariance functions (42).

Mathematically, the standard formulas that guide the posterior mean $\mu(x^*)$ and variance $\sigma^2(x^*)$ are given by:

$$\mu(x^*) = \mathbf{k}_*^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}$$

$$\sigma^2(x^*) = k(x^*, x^*) - \mathbf{k}_*^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*$$

Where:

- $\mathbf{K}$ is the covariance matrix computed over the training inputs.
- $\mathbf{k}_*$ is the covariance vector between the test input and training inputs.
- σ_n^2 represents the noise variance in the observations.

This posterior distribution provides both a prediction (via the mean) and an uncertainty measure (via the variance). These dual components are crucial in BO frameworks, as they allow for a balance between exploration (sampling where the model is uncertain) and exploitation (sampling where the predicted objective is favorable) (21) (42).

In the context of hyperparameter optimization, GPs provide an efficient way to model the objective function with minimal function evaluations, particularly when evaluating the objective is computationally expensive. The GP guides the search by identifying promising regions of the hyperparameter space. This allows the optimization process to converge faster with fewer evaluations (46).

2.3.2 Expected Improvement (EI)

Expected Improvement (EI) is one of the most widely used acquisition functions in BO, particularly due to its effectiveness and analytical tractability when combined with GP priors. The core idea behind EI is to balance exploration and exploitation by quantifying the expected value of improvement over the best observed outcome so far. Formally, if f is modeled with a GP and f_{best} denotes the best function value observed, then the EI at a candidate point x is defined as the expected amount by which $f(x)$ exceeds f_{best} on the GP posterior (21) (31). This formulation ensures that EI favors points with high predicted means (exploitation) and/or high predictive uncertainty (exploration), which makes it particularly well-suited for optimizing expensive black box functions, thus opening an avenue for applying them onto metaheuristics.

Mathematically, assuming a GP posterior at x with mean $\mu(x)$ and standard deviation $\sigma(x)$, the expected improvement is given by:

$$\text{EI}(x) = (\mu(x) - f_{\text{best}})\Phi(z) + \sigma(x)\phi(z)$$

Where:

- $z = \frac{\mu(x) - f_{\text{best}}}{\sigma(x)}$
- Φ is the standard normal cumulative distribution function.
- ϕ is the standard normal probability density function.

The use of EI in BO has been shown to be highly effective particularly in ML model tuning, due to its closed-form computation and principled uncertainty handling (10) (46). However, it is worth noting that EI, like other acquisition functions, can become shortsighted, optimizing short-term gain, and may benefit from hybridization or exploration penalties in high-dimensional or noisy settings (44).

This observation is directly relevant to the objectives of this thesis, given the stochastic nature of Genetic Algorithms (GAs). GAs rely on random processes such as initialization, selection, and mutation, resulting in inherent variability in their outputs. Consequently, when GAs are treated as objective functions for HPO, they introduce noise. Frazier (2018) (21) explicitly notes that BO is well-suited for such noisy objectives, provided the surrogate model, such as a GP, accounts for observation noise. This makes modeling noise not only appropriate but also an essential aspect of effective surrogate reliant optimization in this context, hence being worthy to include in the experiment.

2.3.3 Tree Parzen Estimator (TPE)

Another widely used approach is the Tree Parzen Estimator (TPE). The TPE replaces the surrogate model component of BO with a model based on density. The algorithm splits the observations to two subsets, based on the quality of the solutions. This is done using a predetermined boundary, also known as the top-quantile, which is recomputed at each iteration using a separate quantile selection algorithm that depends on the incumbent number of observations. Different quantile calculation strategies exist and influence the exploration-exploitation trade off (eg. linear and squared scaling) (6).

For a set of observations $\mathcal{D}$, with x , and the following objective function $y = f(x) + \epsilon$ where ϵ represents noise, the disjointed subsets are denoted as $\mathcal{D}_{\text{good}}$ for the better

group and $\mathcal{D}_{\text{bad}}$ for the worse group. Using Kernel Density Estimators (KDEs), which are non-parametric methods that estimate the Probability Density Functions (PDFs), TPE models the densities for each respective subset and subsequently computes the density ratio:

$$r(x \mid \mathcal{D}) = \frac{p(x \mid \mathcal{D}^{(l)})}{p(x \mid \mathcal{D}^{(g)})}$$

Where:

- $p(x \mid \mathcal{D}^{(l)})$ is the density of group $\mathcal{D}_{\text{good}}$
- $p(x \mid \mathcal{D}^{(g)})$ is the density of group $\mathcal{D}_{\text{bad}}$

The ratio serves as the acquisition function in TPE, assigning high values to configurations more likely to yield good objective values, thereby guiding the optimization toward promising regions (6).

The KDEs are built using kernel functions, typically Gaussian for numerical variables, as in our case. Each kernel is parameterized by a bandwidth b_l and b_g for the good and bad groups, respectively. These bandwidths are control parameters that tune the smoothness of the PDF estimates, which in turn govern the exploration and exploitation trade-off, while also helping to manage noisy observations. A minimum bandwidth constraint b_{min} , scaled accordingly, is enforced to prevent overfitting and excess exploitation (6).

To ensure numerical stability and avoid density overflows, TPE implementations often use magic clipping, a heuristic that clips extreme density values to a maximum threshold. This prevents outliers or tightly packed samples from dominating the density ratio (29). TPE also includes a weighting mechanism that assigns varying importance to samples based on their relative contribution to the kernel densities. Several weighting schemes exist, including uniform, decay-based and expected improvement weighting (6).

In the TPE framework, the surrogate model is the pair of KDEs for each subset, and the acquisition function is the density ratio. Thus, TPE can be viewed as a variant of Bayesian Optimization. The diversity of configuration that the TPE offers raises the question of which design choices are most appropriate. The reasoning behind the decisions made in this aspect will be specified on Sect. 4.2. Its modified approach to BO and the strong support it receives in AutoML libraries make it a strong candidate for experimentation.

Chapter 3 Literature Review

3.1 Introduction to the Literature Landscape

The optimization of algorithmic performance through hyperparameter tuning has long been a focal point in both ML and operations research (OR). Both domains must carefully handle the parameterization to gain sufficient results. Within ML, the rise of AutoML systems has significantly lowered the barrier to achieving high performance by automating key steps of the pipeline, particularly HPO (29). In parallel, metaheuristic algorithms gained prominence in OR for solving complex, NP-hard combinatorial problems. These algorithms, however, often require extensive tuning of hyperparameters to achieve optimal results (15) (14).

Despite parallel progress in both communities, the intersection of AutoML and metaheuristic tuning remains underexplored, especially from an experimental standpoint. Most applications of AutoML have been limited to traditional supervised learning tasks (19) (8), while tuning of metaheuristics continues to rely on heuristic or partially automated methods such as grid search or adaptive mechanisms (7) (36). Moreover, few studies systematically benchmark AutoML HPO methods, such as Bayesian Optimization (BO), on real world metaheuristic configurations, especially for structured scheduling problems.

This thesis contributes to filling this gap by applying AutoML techniques, particularly Bayesian-based HPO strategies, to the hyperparameter tuning of a GA for a weighted tardiness scheduling problem in an unrelated parallel machine setting. By treating the GA as a black box objective and leveraging open source AutoML libraries, the study offers a practical and generalizable framework. Importantly, the goal is not to produce an exhaustive survey or universal benchmark, but to highlight the viability, performance or resource efficiency gains of this integration in a complex optimization scenario.

This literature review surveys prior work on manual tuning, HPO frameworks, the limited but emerging efforts to apply AutoML in the optimization and metaheuristics domain and the research gap in current literature for this topic.

3.2 Manual Tuning and the Motivation of Automation

Manual tuning has traditionally been the default strategy for calibrating metaheuristic algorithms. It often depends on expert intuition, informal experimentation, and iterative trial-and-error, making it time-consuming and computationally costly. Moreover, it lacks reproducibility and does not scale well across different problem instances (15). Without a systematic methodology, manual tuning is prone to suboptimal configurations, particularly in high-dimensional or noisy search spaces, common in combinatorial optimization. Given the inherent randomness of metaheuristics, this adds further complexity to the tuning process.

Lobo et al. (2007) (35) point out that even structured approaches like design of experiments (DoE) require numerous runs to yield moderately effective configurations. DoE uses statistical analysis to understand how the parameters (inputs) affect outcomes (outputs) and how those parameters interact. Nevertheless, These methods are prone to a combinatorial explosion as the number of parameters increases, and they often overfit to specific problem instances, limiting broader applicability. A combinatorial explosion refers to the exponential growth in the number of possible combinations as more variables or parameters are added. A fixed tuning strategy rarely fits the needs of every instance in metaheuristic problems, behavior can vary widely. At the same time, manual trial-and-error remains inefficient and difficult to work with in applications used in the real world.

Birattari et al. (2009) (7) further critique the ad hoc nature of many manual and partially automated approaches, which often result in inconsistencies due to factors like random seed variability. Although each run may follow the same logic, different seeds can lead to varied outcomes, especially in stochastic processes. This variability is important to measure and control, as it affects reproducibility, stability, and the reliability of reported results.

Manual tuning also struggles with balancing trade-offs between multiple objectives (eg. runtime and solution quality), often focusing on a single metric without structured mechanisms like Pareto front exploration, which facilitates the discovery of multiple optimal solutions by leveraging trade-offs between conflicting objectives (20). Addi-

tionally, it is static and poorly suited to dynamic or evolving problem settings (15).

From a statistical standpoint, manual approaches are frequently based on single or few runs, offering little insight into confidence in outcomes (7) (28). Results are vulnerable to human error and subjective bias, reducing consistency across studies (15). Finally, techniques like grid search or DoE struggle with complex or conditional parameter spaces and fail to capture interdependencies (5) (28). More liabilities of Grid Search and DoE techniques will be addressed in the next section (3.3).

These limitations culminate the need for systematic, scalable, and generalizable solutions, independent of problem uniqueness, that can replace manual tuning in modern optimization contexts.

3.3 Capabilities and Impediments of HPO

Metaheuristics typically include parameters that are independent of the problem formulation and qualify as hyperparameters because they are externally defined settings that govern the algorithm’s behavior but are not learned from data during execution (18). Their influence on convergence speed, solution quality, and computational cost makes them ideal candidates for automated HPO. Traditional HPO methods, such as Grid Search and Random Search, encompass structural simplicity, yet this comes with major limitations. Grid Search, while exhaustive, quickly becomes computationally infeasible in high-dimensional spaces and often overlooks optimal configurations due to its stiff discretization (5). Furthermore, Grid Search faces similar limitations to manual tuning, as the defined contents of the grid still rely on expert intuition. Random Search offers modest improvements in exploration efficiency yet remains relatively uninformed, failing to incorporate feedback from past evaluations to guide the search.

More advanced approaches, such as Sequential Model-Based Optimization (SMBO), introduced a significant leap by integrating surrogate models with adaptive sampling strategies. Notable implementations include SMAC and F-Race (28) (7), both of which substantially reduce evaluation cost compared to brute force methods. SMAC is a variant of Bayesian Optimization that employs a Random Forest surrogate and is specifically designed to handle mixed type hyperparameters. F-Race (and its extension iF-Race) uses statistical racing to evaluates multiple configurations for different problem instances, eliminates underperforming ones early through statistical tests, and concentrates resources on the most promising candidates. Nonetheless, such tools are

often tailored to specific domains, difficult to configure, particularly when applied to noisy, black box functions such as those found in metaheuristic optimization.

To overcome these gaps, recent years have seen the emergence of versatile AutoML libraries that encapsulate advanced HPO strategies and are friendly to developers from a usability standpoint. Optuna (1), BoTorch (4), and Auto-sklearn (19) are among the most widely adopted frameworks, each offering tailored strengths. Optuna provides an intuitive API, dynamic search space definition, and strong native support for pruning via asynchronous Successive Halving, making it suitable for mid scale experimentation in noisy environments. BoTorch, by contrast, focuses on research grade Bayesian Optimization, enabling the design of custom acquisition function and constrained optimization strategies using a PyTorch backend. Auto-sklearn integrates AutoML pipelines for classical ML tasks and automates algorithm selection but is less flexible for algorithmic configurations outside its predefined scopes leading to its exclusion from this study’s experiment.

While these tools significantly reduce the burden of manual tuning, they are not without limitations. Optuna, for example, lacks efficient built-in support for optimization with multiple objectives and requires manual scripting to handle deeply conditional parameter structures. Nonetheless, it offers implementation simplicity, reducing the obstacle for operations researchers to apply it to their own work without extensive ML expertise, a core contribution this thesis seeks to demonstrate.

In contrast, BoTorch demands familiarity with probabilistic modeling and PyTorch, creating a barrier for non-specialists. However, it compensates by offering reduced customization overhead and native support for advanced techniques such as the Knowledge Gradient acquisition function, confirming its suitability for research grade optimization tasks. An advanced framework leveraging the built-in capabilities of BoTorch could serve as a promising direction for future research. One persistent challenge across optimization libraries is the handling of highly stochastic objectives, which often necessitate repeated evaluations and averaging to reduce noise. This process not only increases computational cost but may still fail to sufficiently stabilize noisy performance signals (20).

In sum, while these modern AutoML frameworks represent a critical advance toward addressing the inefficiencies of manual tuning, particularly lack of scalability, inability to balance trade-offs, and poor adaptability, their effective application to metaheuristic optimization still requires thoughtful configuration and awareness of their inherent

constraints. This thesis employs Optuna as the exemplary framework, selected for its usability. The rationale behind the selection and the specific role in the experimental design is detailed in Chapter 4.

3.4 Applications of HPO and AutoML in Optimization and Metaheuristics

As was stated in the introduction, whilst automated HPO strategies are already established in ML, its application to optimization algorithms and metaheuristics remains relatively obscure. In recent years, the notion of treating algorithm configuration as a black box optimization task has gained a bit of traction.

Early efforts relied primarily on manually coded heuristics or models tailored to specific algorithms. For example, Misir and Sebag (2013) (38) introduced a hybrid method using surrogate modeling to combine algorithm selection and parameter for a specific type of problem (Boolean SAT or MaxSAT). This approach assumes the performance patterns are always simple and predictable, which is not true for all problems. Also, when a new problem show up, their model needs to be built from scratch each time. This makes it hard to apply in bigger or more varied settings. In this thesis, the approach is more flexible and can handle different problem types without needing redesigning each time.

A more recent and relevant example is provided by Awad et al. (2020) (3), who employed an AutoML framework to tune parameters in Ant Colony Optimization (ACO). Their results demonstrated improved convergence and performance across synthetic scheduling benchmarks. However, the study did not utilize modern hyperparameter optimization methods, nor did it implement a modular pipeline or address the challenges posed by noisy tasks.

Other notable studies include Calandra et al. (2014) (13), who used Bayesian Optimization to tune gait control policy parameters in robotic control tasks under uncertainty, and López-Ibáñez et al. (2016) (36), who applied the irace tool to fine tune metaheuristics like Iterated Local Search. While these studies showed the benefits of automated parameter tuning, both of them relied on neatly designed parameter spaces adjusted to the needs of their domain of practice. They did not explore noisy or complex parameter spaces, as the ones in this work.

More closely related to the task of this thesis, Rüther and Rieck (2024) (43) applied Bayesian Optimization to tune a Grouping Genetic Algorithm (GGA) for real pickup

and delivery problems. Their study showed significant gains in solution quality by treating the GGA as a black box, but focused on routing tasks, not scheduling. Similarly, Kianpour et al. (2023) (32) proposed a Genetic Algorithm supported by Design of Experiments (DoE) to minimize tardiness in unrelated parallel machine scheduling. However, their method used static tuning strategies and did not leverage black box optimization or integrate AutoML techniques.

What distinguishes the present work is its emphasis on using modern AutoML toolkits (Optuna) to automatically tune a GA in a computationally intensive and real scheduling problem. The task involves weighted tardiness minimization on unrelated parallel machines with sequence-dependent setup times, a setting that is both NP-hard and industrially relevant (47). By wrapping the GA as a black box objective and embedding it in a reproducible AutoML pipeline, the method supports advanced HPO techniques, while allowing for stochastic evaluations, conditional parameter structures, and adaptive resource management.

This integration fills a notable research gap: despite the growing availability of powerful HPO frameworks, their application to metaheuristic algorithms, particularly in complex scheduling scenarios, remains underexplored. Existing studies often rely on custom solutions, focus on low-dimensional or idealized benchmark problems, and lack extensibility to real world, noisy, and computationally intensive tasks. In contrast, this work demonstrates a fully automated, modular, and generalized HPO pipeline, applied to a high stakes industrial scheduling problem. It offers not only empirical validation but also a simple and transferable framework that can be adapted to other black box metaheuristics with minimal effort, bridging a methodological divide between ML automation and classical operations research. At last, the experiment benchmarks the results of two usable methods to test their suitability in different instance settings. The constituents of this work are showcased in more detail in the following chapter.

Chapter 4 Experimental Design

This chapter constitutes the experimental framework used to benchmark AutoML HPO strategies against manual tuning. It begins with the adapted implementation of the Genetic Algorithm (GA), followed by a modular AutoML wrapper, the dataset setup, and the evaluation pipeline. Each component is explained with enough detail to ensure reproducibility. Moreover, the HPO configuration logic, including search space definitions, stopping criteria, and trial budgets is detailed. Finally, the chapter pertains visuals and tables to improve overall comprehension and concludes by outlining the assumptions and limitations that follow this experiment.

4.1 Genetic Algorithm Implementation and Adaptation

The GA used in this thesis is adapted from the implementation proposed by Avgerinos et al. (2023) (2), originally developed to accost the minimization of weighted tardiness in unrelated parallel machine scheduling environments with sequence-dependent and resource-constrained setups. The core evolutionary logic remains intact, including chromosome generation, population based search, selection based on fitness, and the application of crossover and mutation operators. However, the constraint programming component, specifically addressing resource constraints in setups, was removed to reduce computational overhead and to better isolate the impact of hyperparameter tuning. Even with this simplification, the problem continues to exhibit NP-hard complexity.

The GA follows a standard evolutionary approach: job sequences (chromosomes) are randomly initialized and refined over generations. Each chromosome is scored via a fitness function computing total weighted tardiness, considering all job-machine and setup constraints. Top individuals are retained each generation, with offspring produced via probabilistic operators. Candidates are inserted into the population only if they outperform the worst current solution, with the threshold updated dynamically.

For more granularity surrounding the GA, the reader is referred to Avgerinos et al.

(2023) (2), which provides detailed pseudocode and the rationale behind the specific problem.

4.2 Dataset

To simulate realistic conditional scheduling scenarios, synthetic instances were generated, again by following the benchmark creation method of Avgerinos et al. (2023)(2). The generation process accounts for job heterogeneity, sequence-dependent setup durations, and varying deadline constraints. As their original work introduced the benchmark class, the present study adopts the same construction logic without modification. The data generation was handled through Python scripts identical to the original benchmark logic.

The dataset includes 24 instances across four problem sizes:

- 25 jobs $\times$ 5 machines
- 50 jobs $\times$ 5 machines
- 50 jobs $\times$ 10 machines
- 100 jobs $\times$ 10 machines

Each group contains **six instances**, generated with different random seeds to ensure diversity. These instances represent medium and larger problem cases with considerable computational costs. The experiment aims to examine scenarios where manual tuning becomes either suboptimal or even impractical. Smaller instances can often be rerun or tuned manually at relatively low cost, making them less critical targets compared to problems with larger dimensions. Larger problem sizes were not examined, due to the thesis deadline (ironically!) and low hardware capabilities.

The instance generator is controlled by three structural parameters: **setup intensity** (α), **deadline tightness** (τ), and **deadline heterogeneity** (ρ). The parameter α governs the severity of sequence-dependent setup durations. When $\alpha=0$, setups are mild ($0.1\text{--}0.5\times$ the processing time); $\alpha=1$ introduces moderate setups ($0.5\text{--}1$), while $\alpha=2$ corresponds to heavy setups with randomized values between 5 and 25. The parameter τ controls the overall tightness of job due dates, with higher values indicating

stricter deadlines. Finally, ρ manages the variance in deadlines across jobs, introducing heterogeneity in temporal constraints. In all instances used, ρ is fixed to 0.2.

This structural parameterization enables systematic analysis of optimization performance across increasingly constrained scenarios. Once more, the reader is alluded to Avgerinos et al. (2023) (2) for more details about data generation, since the process is kept identical.

4.3 AutoML Wrapper and Optimization Pipeline

To enable systematic and repeatable hyperparameter optimization across different AutoML strategies, a flexible wrapper was developed around the GA. This wrapper automates the entire tuning workflow, from configuration sampling to execution and evaluation, allowing seamless integration of multiple HPO methods with minimal manual intervention. It was designed to ensure both reproducibility and scalability for experimentation across diverse scheduling instances. This scalability potential is particularly useful for future research, in which more advanced fulfillment may be tested. The implementation leverages **Optuna**, used for standard BO variants (TPE and GP+EI), offering efficient out-of-the-box samplers and pruning.

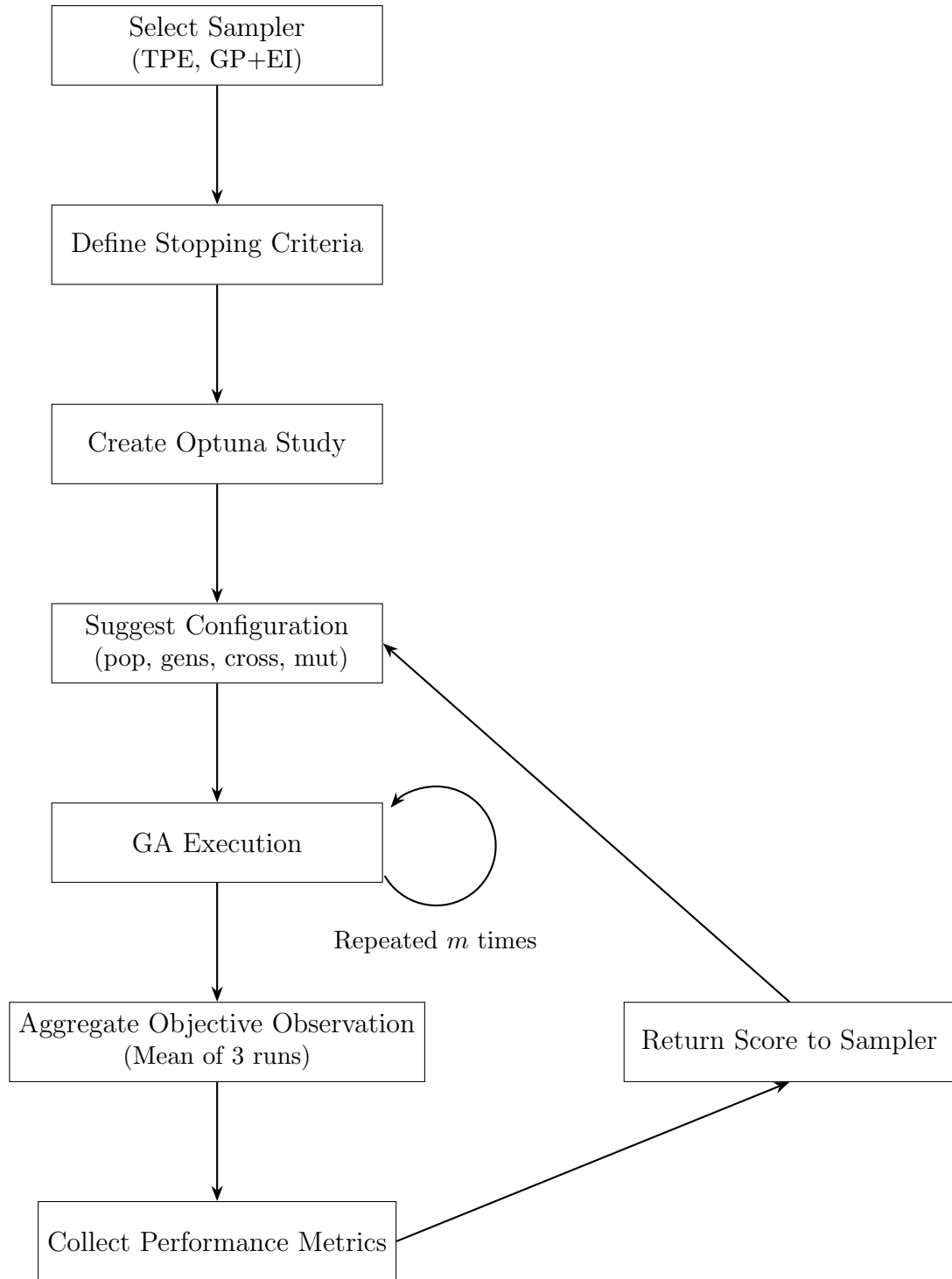
The wrapper exposes a centralized **control flag** to switch between different optimization strategies. Based on the selected method, it dynamically: Calls the correct functions associated to the related library, loads the corresponding sampler and pruner (if it exists), applies a shared trial budget, configures logging and tracking, and manages consistency in parameter search space.

Each sampled configuration is evaluated through **three independent GA runs** to mitigate noise. The final objective observation for a configuration is computed as the **mean weighted tardiness** in those three runs. This approach mirrors the repetition count used in the original baseline study, ensuring that the comparisons are statistically fair. A high level visualization of the wrapper pipeline is shown in Figure 4.1, with n representing the maximum trials allowed and m being the repetition count.

The runtime metrics and all preprocessed outputs are collected for each instance and saved for the processing required to produce the aggregated metrics and benchmarking metrics. Sect. 4.5 details both classes of metrics and the motivation behind their selection.

Finally, adding new optimization methods requires only a small extension to the sampler dispatch logic and objective definition. Each method introduced can also be

customized differently without hindering the overall process. This makes the framework not just suitable for this thesis, but also adaptable for future experiments with different metaheuristics or domains.



Repeat for n trials or until stopping criteria are met

Figure 4.1: Structured flow of the AutoML-GA optimization pipeline

4.4 HPO Strategy Configuration

To enable meaningful comparison between HPO strategies, a consistent yet flexible configuration scheme was applied across all methods. This section details the chosen search spaces, optuna samplers, pruning strategies and the stopping criteria mechanisms, highlighting how each was tailored to the nature of the underlying optimization logic. The objective is to maintain fairness across experiments while leveraging the strengths of each AutoML technique.

4.4.1 Search Space Configuration

The original configuration of the GA that was provided utilized a fixed grid search comprising a limited $2 \times 2 \times 2 \times 2$ configuration set. Specifically, in the provided implementation’s population size was set to either 150 or 200, generations to 200 or 250, crossover rate to 0.5 or 0.75, and mutation rate to 0.15 or 0.30. Therefore, for each instance it inspects the same 16 configurations of parameter sets. Even if they can offer targeted exploration, such grid based methods entail several limitations, as already clarified in Sect. 3.3. Notably, the most relevant weakness is the uniform application of the same grid across all problem instances, disregarding the variability between them.

In contrast, the design of this study adopts a continuous search space to be navigated via Bayesian Optimization methods, allowing probabilistic models to adaptively explore and exploit promising regions without excessive redundant evaluations. An important improvement for stochastic problem spaces.

In detail, the search space was extended to cover the following range of values for each hyperparameter:

- Population size: [150, 400]
- Generations: [200, 400]
- Crossover Rate: [0.3, 0.8]
- Mutation Rate: [0.05, 0.5]

Population size and number of generations are treated as **integer-valued continuous** parameters with a step size of two, allowing smoother exploration without

unnecessary resolution. Conversely, crossover rate and mutation rate are modeled as **continuous floating-point** variables, enabling delicate tuning across their ranges.

These ranges were selected based on recommendations from the research team, empirical rationale and existing literature, which suggests that increased population size and generation count can improve the convergence quality and solution diversity in genetic algorithms, albeit with considerable computational cost (14).

4.4.2 Additional Configurations

As shown in Sect. 4.3, the experimental pipeline incorporates two variants of Bayesian Optimization (BO) as the examined HPO strategies. The variants differ in their surrogate models, acquisition functions, and sampling behaviors, each influencing in its own way: the balance of exploration vs exploitation, convergence properties, and computational cost. These selected methods reflect both theoretically grounded and practically established approaches available in modern AutoML toolkits. As a pruning method, the default MedianPruner function, natively supported by Optuna, was enabled for both of the strategies.

Stopping criteria

The evaluation was performed under a fixed trial budget and identical early stopping criteria to ensure fair benchmarking. The trial budget for the Optuna study was set to 25 per instance. If no improvement has been found after five trials, the algorithm is stopped early. As discussed, 16 configurations are evaluated uniformly in the manually tuned grid search, the baseline of this study. Hence, motivating the decision to slightly increase the upper bound of configuration evaluations to account for worst case scenarios, whilst maintaining tolerable computational efficiency. However, this does not increase cost intensity compared to the baseline, as this milestone is improbable to reach and even if that were the case, undocumented trial-and-error effort still would result in higher implicit cost. Pilot runs indicated that improvements generally plateaued after 3-5 iterations of non-improvement without exaggerated computation, validating this threshold choice empirically.

Another stopping criterion used was a per instance time limit applied exclusively to the AutoML methods. This limit can be adjusted based on the user’s available time-related resources or, as in this study, aligned with the approximate execution time per instance of the baseline method. The time limit was scaled according to problem

size, as instances with higher dimensions require longer runtime. For each problem size, the approximate execution time range of its corresponding baseline instances was first determined. Based on this range, a reasonable time limit was set, carefully chosen to balance fairness and the potential for meaningful improvement. The overarching goal was to demonstrate that AutoML methods can still yield improved performance even under stricter time constraints, specifically, using time limits shorter than the fastest execution time recorded by the baseline method. This rationale was enforced in the largest and most complex problem sizes. In the shorter ones, the budget was hovering around 1000 seconds, due to some configurations needing more time to finish before the budget is activated. The 1000 second bound was nearly double the time that the baseline exhibited. This is allowed because of the implicit cost phenomenon, better described in Sect. 5.2.

Method Configuration

The first method (**GP+EI**) employs a Gaussian Process (GP) surrogate combined with Expected Improvement (EI) as the acquisition function, implemented via Optuna’s native **GPSampler**, which includes EI as the default acquisition function option. No additional configurations were applied, to preserve this method’s simplicity and leverage Optuna’s easy-to-use interface. GP+EI is the most standard BO variant and, while it attempts to balance the exploration-exploitation tradeoff, it may lean more toward exploitation compared to other methods. The simplicity of this approach is illustrated in Algorithm 1. One of the research objectives of this thesis, mentioned in Sect. 1.2, is to bridge the traditional optimization domain with state-of-the-art ML techniques without requiring deep machine learning expertise, making simplicity a key consideration.

Algorithm 1 GP+EI Configuration (Optuna)

- 1: Initialize Gaussian Process Sampler
 - 2: Use Expected Improvement (EI) as acquisition function
 - 3: **return** GPSampler()
-

The second method utilizes Tree-structured Parzen Estimators (**TPE**), implemented via Optuna’s native **TPESampler**. As an alternative to GP+EI, research-approved configurations were included based on the recommendations of Watanabe et al. (2022) (48) to improve performance. The choices draw inspiration from the classic Hyperopt setup: KDEs were weighted using EI as the weighting scheme, and the γ parameter,

used for splitting observations into good and bad groups, is scheduled linearly to better suit the low-dimensional search space. Magic clipping is enabled to stabilize extreme density ratios during sampling, and prior distributions are incorporated to prevent convergence to local optima in early sampling phases. These modifications, inspired by literature and shown in Algorithm 2, help bridge the gap between experimental tuning and practical application, while also demonstrating how minor enhancements over defaults can perhaps improve method stability in noisy or constrained domains.

Algorithm 2 TPE Configuration with Enhanced Settings

```

function LINEARGAMMA( $n$ )
  return min(25, max(1,  $\lfloor 0.1 \cdot n \rfloor$ ))
end function

Initialize TPE Sampler with:
  consider_prior  $\leftarrow$  True
  prior_weight  $\leftarrow$  1.0
  consider_magic_clip  $\leftarrow$  True
  consider_endpoints  $\leftarrow$  False
  n_startup_trials  $\leftarrow$  10
  multivariate  $\leftarrow$  True
  group  $\leftarrow$  True
  gamma  $\leftarrow$  LinearGamma
  weights  $\leftarrow$  EI-based KDE weights
return TPESampler(...)

```

Further information about the interior design of these methods can be found in Sect. 2.3.

4.5 Selected Metrics

To evaluate the performance of the tuning methods, two layers of metrics were collected. The first layer comprises metrics obtained during runtime. The second layer involves aggregation performance metrics across all trials per method, including deviation ratios and comparative deltas against the baseline. This way both algorithm inspection and benchmarking were supported.

In Sect. 4.3., it is highlighted that each configuration attempt was evaluated thrice to combat stochasticity. This design was prevalent across all optimization methods, including the manually tuned baseline. The mean solution value from these three runs served as the scalar objective observation fed into the AutoML frameworks. All three

output values from the best performing configuration were stored to carry out deeper analysis.

Alongside the primary performance metric (**total weighted tardiness**), each experiment stored other auxiliary data as well. This included:

- **Number of solutions evaluated**
 - Represents the total search effort per configuration, helping to quantify both exploration intensity and computational budget.
(Computed as: population size $\times$ number of generations)
- **Best performing hyperparameter configuration**
 - Identifies the parameter set responsible for the lowest observed weighted tardiness, providing insight into which configurations can provide satisfactory results, thus enhancing reproducibility.
(Includes: population size, generations, crossover rate, mutation rate)
- **Total execution time per instance**
 - Quantifies the overall computational cost of the tuning process, allowing for method comparison from a perspective of resource efficiency and, to some degree, scalability.
- **Number of configurations tested**
 - Reflects how efficiently each method navigates the search space within a fixed trial budget, indicating optimization effectiveness.
- **Individual GA runtimes**
 - Captures the execution time of each run, useful for detecting anomalies and assessing performance variability due to stochastic behavior.

Obtaining the runtime metrics enabled the additional processing required to generate the second layer of comparative metrics. The first step involved aggregating total weighted tardiness values by instance, allowing the calculation of the stochastic average and standard deviation for each method’s best performing configuration. Based on these aggregated values, the benchmarking metrics presented in Table 4.1 below were derived.

Metric	Purpose	Formula
Performance Delta (Δ_{Sol})	Measures the actual deviation between the baseline and AutoML mean performance. Negative values indicate improvement.	$\Delta_{\text{Sol}} = \mu_{\text{Baseline}} - \mu_{\text{AutoML}}$
Error Rate (%)	Shows relative improvement as a percentage of the baseline. Negative values reflect improvement.	$\text{Err \%} = \left(\frac{\Delta_{\text{Sol}}}{\mu_{\text{Baseline}}} \right) \times 100$
Standard Deviation Ratio	Compares variability of AutoML versus baseline. Values <1 indicate that AutoML methods exhibited less variability.	$\text{Std Ratio} = \frac{\sigma_{\text{AutoML}}}{\sigma_{\text{Baseline}}}$
Relative Standard Impact (RSI)	Assesses if improvement outweighs noise. Values <1 imply stable results, with values closer to 0 being the ideal.	$\text{RSI} = \frac{\sigma_{\text{AutoML}}}{ \Delta_{\text{Sol}} + \epsilon}$
Solutions Evaluated Ratio	Reflects the efficiency or exploration capability of the method relative to the baseline	$\frac{\text{Sols}_{\text{AutoML}}}{\text{Sols}_{\text{Baseline}}}$
Execution Time Ratio	Captures the time per instance between AutoML relative to the baseline.	$\frac{t_{\text{AutoML}}}{t_{\text{Baseline}}}$
Configurations Tested Delta	Highlights the difference in number of configurations evaluated by each method.	$C_{\text{AutoML}} - C_{\text{Baseline}}$

Table 4.1: Benchmarking metrics used for AutoML performance evaluation

4.6 Assumptions

To ensure a tractable and coherent experimental framework, several assumptions were adopted throughout the design and execution of the study. First, it was assumed that averaging the performance across three stochastic GA runs per configuration would sufficiently smooth the impact of randomness, thus yielding a reliable objective signal for the samplers. Although the GA is inherently stochastic, from the perspective of the

AutoML optimizer, it was treated as a pseudo-deterministic black box function with averaged outputs, allowing consistent feedback into the optimization loop.

It was also assumed that the computational effort could be appropriately quantified by proxy metrics such as population size multiplied by generations, and that execution time per instance was indicative of runtime complexity and resource demand. The cost model implicitly assumed uniform resource usage per configuration and did not consider deeper system-level performance factors.

Lastly, the tuning methods were evaluated on a single hardware platform with fixed resources, and no external parallelization or distributed processing was introduced, ensuring that comparisons were strictly based on algorithmic behavior rather than system-specific acceleration.

4.7 Limitations

As with all experiments, this study has limitations that obstruct its scope. The decision to average over three stochastic runs per configuration helps reduce noise but may not be sufficient to fully eliminate the effects of randomness in GA outcomes. Outlier runs may still influence the aggregated metrics, and deeper resampling strategies could offer more stable evaluations in future work. Moreover, the dataset only comprised of 24 instances, with six instances belonging to each problem size, thus outlier instances have a deeper effect to insight objectivity due to the limited dataset.

All experiments were conducted on a laptop grade CPU, which imposed limitations on runtime and constrained the breadth of experimentation. With access to more powerful computational resources, much larger problem instances could be systematically analyzed, enhancing generalizability and the practical applicability of the findings. Additionally, larger-scale experiments would facilitate more extensive resampling strategies and allow more objective evaluation for each method, strengthening the study’s comprehensiveness. The study focused exclusively on Genetic Algorithms, without extending to other metaheuristic paradigms such as Ant Colony Optimization, Simulated Annealing, or Tabu Search. While this constraint maintains focus, it also highlights that the aim of this thesis is not to deliver exhaustive generalization, but rather to demonstrate the potential of AutoML when applied to metaheuristic tuning in classical optimization settings. The breadth of experimentation can be further fulfilled in future work with the proper computational resources.

Although the runtime of each individual GA run was recorded, these measurements

are somewhat distorted by overhead introduced by Optuna’s orchestration processes (e.g., trial queuing, sampler interaction, and Python process management). Consequently, they do not reflect the pure runtime of the GA itself and were therefore excluded from core benchmarking. Instead, emphasis was placed on the total instance-level execution time, which offers a more consistent and comparative signal across methods. Still, potentially useful insights obtained from this metric were disregarded.

The hyperparameter search space, particularly the selected ranges, was manually defined based on guidance from the original GA developer and refined through early design iterations. While this design aligns with expert intuition and captures relevant regions, it still relies on fixed boundaries. Nevertheless, it offers a broader and more expressive space than traditional manual tuning and enables meaningful exploration by AutoML methods. Even so, fixed space design may introduce mild bias, as performance can depend on how well this space aligns with high performing regions.

Each AutoML method was executed once per experimental setup. While individual GA configurations were averaged across multiple runs, the tuning process itself was not repeated under different random seeds. This leaves room for trajectory variance, especially for methods like TPE or GP-based samplers. Additionally, the configurability of Bayesian Optimization methods is somewhat constrained by the framework used. For example, Optuna does not expose full control over surrogate models or acquisition functions in all cases, which may limit achievable performance due to reliance on internal heuristics.

Finally, the study does not include a formal sensitivity analysis to evaluate how solution quality responds to small perturbations around high-performing configurations. Such an analysis would be valuable in assessing the local stability of discovered optima.

Chapter 5 Results

This chapter presents the experimental results comparing three hyperparameter tuning strategies: the manually tuned Grid Search baseline and two AutoML methods, Gaussian Process with Expected Improvement (GP+EI) and Tree-structured Parzen Estimators (TPE). The goal is to assess whether AutoML strategies can improve performance relative to a strong, expert-informed baseline. Results are analyzed across multiple problem sizes and instance characteristics using a defined set of benchmarking metrics, focusing on solution quality, stability, computational efficiency and search space exploration. The general methodology was described in detail in the previous chapter.

5.1 Solution Quality and Variability

The primary indicators of solution quality are the Performance Delta (Δ_{Sol}) and Error Rate, both formally introduced in Sect. 4.5. The aggregated results for these metrics, along with supporting metrics such as *Standard Deviation Ratio*, *RSI*, and *Solutions Evaluated Ratio*, are presented below.

5.1.1 Solution Quality by Problem Size

GP+EI

In the smallest configuration, **25×5**, all six instances demonstrated improved solutions over the baseline, with consistently negative Δ_{Sol} values. However, the scale of these improvements was marginal, the median Error Rate settled at just -0.85% , paired with a median RSI of 0.53. While the method reliably outperformed the base-

line, the modesty of the gains raises important questions about return on investment. Is it justifiable to incur the full computational burden of a search assisted by surrogates when the margin of improvement sits within such low percentage points? The answer is debatable and should also consider computational cost, which will be elaborated upon in a following section (5.2). In low-dimensional problems, where the baseline may already be operating near the frontier of solution quality, diminishing returns from advanced optimization methods may be an unavoidable reality, yet this still merits further investigation.

The **50×5** configuration brought a more nuanced landscape. After removing one problematic instance, with an abnormally low baseline value (total weighted tardiness of 4.3), the remaining instances revealed a more promising trend. The median Error Rate improved to -3.97% , and the median Δ_{Sol} deepened to -478.65 , as expected with the increase in job count. The five instances combined to produce a median RSI of 0.67, which still suggests an evidently stringent optimization process. Yet, the presence of such divergent outliers is revealing: these outliers hint at a potential brittleness in the GP+EI approach, one that might be sensitive to the inherent structure or cost landscape of specific problem instances.

Increasing the number of machines in **50×10** yielded somewhat plateaued benefits. Here, the median Error Rate was -4.27% , less favorable than in 50×5, despite the increase in dimensionality. The spread among individual instances (from -0.62% to -8.28%) and a median RSI of 0.66 suggest that improvements remained stable but not particularly amplified. This stagnation in gains suggests that simply **increasing machine count does not proportionally increase problem complexity** from the perspective of the search algorithm. Unlike job count, which contributes to factorial growth in the sequencing space, machine count seems to exert a more tempered influence. This observation invites further inquiry: if tuning becomes no more difficult with more machines, can we reframe optimization strategies to exploit this asymmetry?

The **100×10** configuration offered the clearest signal of GP+EI’s potential. The median Error Rate dropped to -9.49% , with half the instances exceeding -11% , and two surpassing the -15% threshold (-15.21% and -16.69%). On its face, this is the kind of performance one hopes for in search with surrogates. However, the variability in RSI values paints a more complex picture. While two instances showed highly consistent results ($\text{RSI} < 0.3$) and one achieved moderate consistency (0.84), the remaining three had markedly unstable behaviors, reaching as high as 6.75. Curiously, the most impressive gains aligned with the most reliable runs, suggesting that **when**

the method “locks in”, it can extract deep structural advantages. But when it fails, it does so inconsistently and, occasionally, wastefully. Such disparity signals a deeper issue: search dynamics in high-dimensional spaces may be subject to local attractors or overly greedy exploitation, problems that even a surrogate model calibrated for exploration may struggle to overcome without additional diversity control or strategic restart mechanisms. As already mentioned, GP+EI is slightly leaning towards exploitation, more so than exploration, which perhaps enhances this claim.

Across all problem sizes, one trend remains clear: the solution ratios consistently outperformed the baseline. This uniform improvement, even when modest, signals that GP+EI is systematically enabling more effective traversal of the search space. The method appears capable of uncovering better optima, even in cases where the absolute gains are small, an outcome that hints at its potential utility as a refinement or enhancement layer in larger optimization pipelines.

Still, the presence of outliers, occasional regressions, and highly variable stability suggest that GP+EI is not a universally dependable solution. It is not “plug and play.” Tuning sensitivity, problem-specific features, and stochastic volatility all appear to shape the outcome meaningfully. These results do not undermine its potential, in fact, they highlight it, but they also argue for a more cautious and nuanced deployment. Future work could explore adaptive control of surrogate update schedules, hybridization with heuristic baselines, or embedding robustness criteria directly into the acquisition function to smooth out volatility.

In short, GP+EI shows clear signs of promise, especially in larger and more complex instances. But its success comes with conditions. Understanding *why* it performs well, or why it sometimes falters, is as important as the performance numbers themselves.

TPE

The **25×5** configuration yielded the mildest improvements among all problem sizes. While most instances saw predominantly insignificant gains, reflected in a median Error Rate of just -0.2% , the associated median RSI of 1.11 complicates interpretation. Unlike in GP+EI, where low RSI often accompanied high quality outcomes, TPE here appears less stable even when performance changes are minimal. This may indicate a lack of consistent convergence behavior or an overreaction to early sampling noise. Notably, the median performance delta was only -7.35 , emphasizing the minimal absolute improvement. These results call into question whether TPE meaningfully benefits

small, low-dimensional problems, or whether its more exploratory sampling mechanism results in overfitting noise in shallow landscapes.

With the **50×5** configuration, TPE produced more substantial results. The median Error Rate improved to -3.34% (again without the outlier instance). However, the method’s instability persisted: a median *RSI* of 2.42, suggests large performance swings across replicates. This level of variance could be the byproduct of the TPE’s nonparametric approach, which lacks the regularization mechanisms of Gaussian Process models. While better than the 25×5 case, this configuration hints at a structural sensitivity in TPE’s modeling behavior that may require tuning of kernel density estimation parameters or additional regularization.

For the **50×10** setup, the gains were comparatively muted. The median Error Rate was only -1.91% , despite higher dimensionality, coupled with a median *RSI* of 2.29, supports the hypothesis that the added machine count alone does not increase complexity in a way TPE can exploit.

The most encouraging outcomes emerged again in the **100×10** configuration. TPE achieved a median Error Rate of -7.83% with one instance showing reduction of -35.64% and two others being around 10%. While these figures indicate that TPE is indeed capable of deep improvement in complex search spaces, they must be viewed alongside a median *RSI* of 0.71, which is acceptable consistency. These results suggest that TPE has the capacity to locate valuable optima without being dominated by noise impact.

In summary, TPE presents promising potential, particularly in larger, high-dimensional configurations, though its effectiveness appears more conditional than GP+EI. While it has shown the capacity to generate substantial improvements, this is frequently offset by erratic behavior across instances and problem sizes. Its KDE-based modeling facilitates wide exploration, but this flexibility may come at the cost of convergence reliability, especially in mid scale or structurally volatile settings.

Nonetheless, the method’s capacity to outperform baselines, sometimes dramatically, highlights its inherent value. Rather than being dismissed due to instability, TPE merits deeper refinement.

Table 5.1: Performance metrics across problem sizes for GP+EI and TPE.

		GP+EI		TPE	
Size	Instances	Error Rate (%)	RSI	Error Rate (%)	RSI
25×5	6	-0.85	0.53	-0.20	1.11
50×5	5	-3.97	0.67	-3.34	2.42
50×10	6	-4.27	0.66	-1.91	2.29
100×10	6	-9.49	0.56	-7.83	0.71

5.1.2 Results by α , τ

The parameters α and τ control critical structural aspects of the scheduling instances: α denotes setup time intensity, with higher values reflecting more demanding job sequencing conditions, while τ represents deadline tightness, where higher values indicate more restrictive due dates. The resource constraint parameter ($\rho = 0.2$) remains fixed across all scenarios. To support the analysis, each subsection is accompanied by a summary matrix reporting the aggregated performance metrics. Any detected outliers are marked with an asterisk (*).

Low Setup Intensity ($\alpha = 0$)

Under minimal setup intensity ($\alpha = 0$), both GP+EI and TPE methods achieved limited improvements. When deadlines were moderately relaxed ($\tau = 0.5$), GP+EI and TPE exhibited median Error Rates of approximately -1.96% and -1.33% , respectively, with notable instability (median RSI around 1.6 for both). Tightening the deadline constraint ($\tau = 0.8$) to generate instances, marginally boosted GP+EI performance to a median Error Rate of -2.3% , while also significantly decreasing volatility (median RSI = 0.27). Despite an outlier with an RSI of 5.58, all remaining values were below the 0.3 threshold. TPE demonstrated similar performance improvements, reaching a median Error Rate of -1.84% under relaxed deadlines, while also reducing its variability (median RSI = 0.26), aside from the same outlier instance.

These shifts are further illustrated in Tables 5.2 and 5.3, where no coherent trend emerges across problem sizes. For $\tau = 0.5$, the values fluctuate without clear structure, introducing the observation that relaxed deadlines may induce inconsistent and

somewhat erratic behavior.

Overall, low setup intensity combined with tight deadlines resulted in exploratory gains, with median solution ratios reaching approximately 55% for both methods. When the deadlines relaxed, TPE retained the 55% ratio, while GP+EI dropped to 34%, reinforcing the notion that the latter leans more toward exploitation than exploration.

Table 5.2: Performance metrics for $\alpha=0$, $\tau=0.5$ across problem sizes.

Problem Size	GP+EI			TPE		
	Error Rate (%)	RSI	Sol. Ratio (%)	Error Rate (%)	RSI	Sol. Ratio (%)
25×5	-0.65	0.31	+34.96	-0.18	1.15	+55.36
50×5	+1.38	2.6	+32.91	-0.81	2.51	+55.27
50×10	-3.27	0.58	+53.80	-1.85	2.19	+54.96
100×10	-4.27	6.76	+33.58	-35.65*	0.67	+70.38

Table 5.3: Performance metrics for $\alpha = 0$, $\tau = 0.8$ across problem sizes.

Problem Size	GP+EI			TPE		
	Error Rate (%)	RSI	Sol. Ratio (%)	Error Rate (%)	RSI	Sol. Ratio (%)
25×5	-0.03	0	+9.09	-0.03	0	+56.54
50×5	-3.97	0.29	+52.56	-3.34	0.17	+53.36
50×10	-0.62	5.58	+58.97	-0.33	3.45	+49.07
100×10	-11.53	0.24	+74.36	-9.55*	0.35	+64.78

Moderate Setup Intensity ($\alpha = 1$)

Increasing setup intensity to moderate levels ($\alpha = 1$) led to more noticeable, though still uneven, improvements. With moderately relaxed deadlines ($\tau = 0.5$), GP+EI produced meaningful median gains of -3.52% , excluding an extreme outlier with an uncharacteristically low weighted tardiness of 4.3. The corresponding median RSI of 1.65 still indicates notable fluctuation. TPE yielded stronger median results at -6.07% ,

with slightly better consistency ($\text{RSI} = 1.18$), again excluding the same outlier. However, the corresponding table 5.4 reveals that no consistent pattern can be extracted across problem sizes for either method in this setting, variability remains high, and performance wavers.

Further relaxing deadlines ($\tau = 0.8$) significantly produced sturdier results for GP+EI ($\text{RSI} = 0.55$), while simultaneously maintaining moderate gains (-4.55%). TPE showed similar performance (-4.41%) but with mixed stability. Two of the instances demonstrated clear robustness, with RSI values below 0.3, while the rest exhibited moderately high volatility, with RSI values around 1.8.

Solution ratios reached medians around $50 - 60\%$, reflecting that the method consistently generated a higher number of solutions across generations. These trends, visible in the tabulated results, imply that GP+EI starts to evidently benefit from tightened time constraints when paired with moderate setup demands.

Table 5.4: Performance metrics for $\alpha = 1$, $\tau = 0.5$ across problem sizes.

Problem Size	GP+EI			TPE		
	Error Rate (%)	RSI	Sol. Ratio (%)	Error Rate (%)	RSI	Sol. Ratio (%)
25×5	-1.41	3.16	+58.33	+0.85	1.25	+67.54
50×5	-64.17*	0.97	+54.61	-33.33*	3.25	-43.11*
50×10	-8.28	0.89	+56.52	-8.41	1.10	+45.91
100×10	-3.52	2.32	+67.95	-6.07	1.03	+51.70

Table 5.5: Performance metrics for $\alpha = 1$, $\tau = 0.8$ across problem sizes.

Problem Size	GP+EI			TPE		
	Error Rate (%)	RSI	Sol. Ratio (%)	Error Rate (%)	RSI	Sol. Ratio (%)
25×5	-0.01	51.33*	+64.07	-0.23	1.86	+55.54
50×5	-6.89	0.35	+55.63	-8.28	0.27	+56.29
50×10	-2.20	0.74	+55.56	-2.71	0.24	+59.18
100×10	-7.44	0.28	+48.03	-6.11	1.78	+35.21

High Setup Intensity ($\alpha = 2$)

At high setup intensity ($\alpha = 2$), GP+EI exhibited its most consistent behavior. With moderately tight deadlines ($\tau = 0.5$), GP+EI achieved a strong median improvement of -6.43% and remarkable stability, reflected by a median RSI of 0.52. In contrast, TPE did not follow the same trajectory, reaching a lower Error Rate of -3.27% and exhibiting similar volatility to the moderate intensity, with an RSI of 1.36.

Reinforcing deadline tightness ($\tau = 0.8$) yielded slightly better reliability, with GP+EI achieving a median RSI of 0.48, alongside an acceptable median weighted tardiness of -3.93% , reaching as low as -16.69% in the best case. Conversely, TPE wavered as setup intensity increased, with median RSI values rising from an already volatile 1.36 to 3.31. This trend was partially skewed by an extreme outlier (RSI = 70.32), which does not accurately reflect the method’s typical behavior. Nonetheless, even without the outlier, the overall volatility remained above acceptable thresholds.

Both methods achieved high solution ratios for $\tau = 0.8$; (GP+EI: 60.77%, TPE: 67.45%), yet TPE remained less stable and less effective overall.

For $\tau = 0.5$, solution ratios declined to 34% and 45% respectively, another case where GP+EI demonstrates its exploitative leaning. One outlier instance skewed the distribution with a negative solution ratio of -58.73% , despite achieving a strong Error Rate of -6.21% . This suggests that improvement in solution quality does not always align with broader exploration.

These patterns are cautiously reflected in Tables 5.6 and 5.7, where GP+EI appears considerably more consistent. RSI values remain below 0.8 across all problem sizes, and Error Rates improve steadily with scale, an effect clearly observed in every instance under high setup intensity. In contrast, TPE’s results appear significantly more erratic, lacking a discernible trend and exhibiting elevated volatility throughout.

Table 5.6: Performance metrics for $\alpha = 2$, $\tau = 0.5$ across problem sizes.

Problem Size	GP+EI			TPE		
	Error Rate (%)	RSI	Sol. Ratio (%)	Error Rate (%)	RSI	Sol. Ratio (%)
25×5	-4.41	0.09	+38.25	-4.57	0.11	+56.35
50×5	-6.64	0.67	+29.83	-5.19	2.42	+34.43
50×10	-6.21	0.36	-58.73*	-1.96	2.39	+19.81
100×10	-15.21	0.84	+44.43	-1.62	0.33	+56.54

Table 5.7: Performance metrics for $\alpha = 2$, $\tau = 0.8$ across problem sizes.

Problem Size	GP+EI			TPE		
	Error Rate (%)	RSI	Sol. Ratio (%)	Error Rate (%)	RSI	Sol. Ratio (%)
25×5	-1.05	0.74	+60.65	-0.94	1.07	+68.35
50×5	-2.56	0.68	+55.41	+0.06	70.32*	+70.22
50×10	-5.26	0.11	+60.88	-1.85	3.31	+66.55
100×10	-16.69	0.28	+63.47	-10.22	0.76	+46.12

Overlying Trends and Interpretation

A clear trend emerges: GP+EI demonstrates greater sensitivity to changes in setup intensity (α) and deadline tightness (τ) compared to TPE, exhibiting progressively stronger performance and reduced variability as scheduling constraints evolve. In particular, GP+EI adapts especially well under higher setup intensity, where both performance and stability improve markedly. Additional improvements, yet less impactful, are also observed when deadline tightness is reinforced. In contrast, TPE appears less responsive to these structural adjustments and more dependent on individual problem sizes, as indicated by the results. While it achieved substantial solution ratios, its performance remained inconsistent overall. GP+EI, by comparison, seems to adapt more reliably across diverse configurations.

5.2 Computational Cost

Another important aspect to examine is the computational footprint of these methods relative to the baseline. While results regarding solution quality and variability indicate improvements in minimizing weighted tardiness, a key observation is that these methods typically deliver at least equivalent outcomes in the worst case, and more often yield improvements, barring a few isolated outliers. Thus, if it can be demonstrated that these comparable or improved outcomes are accompanied by a substantial reduction in computational burden, the contribution of the surrogate methods becomes more clearly justified. The analysis in this section specifically aims to investigate whether such computational efficiencies are indeed realized.

The metrics selected for the analysis are the **Instance Execution Time Ratio**, which quantifies, as a percentage, the relative deviation in execution time between the surrogate-based methods and the baseline (ratio > 1 implies higher cost); and the **Configurations Tested Delta**, which assesses whether such deviations correspond to reductions in the number of tested configurations. The execution time ratio metric is visually supported by tables that display its results. Specifically, Table 5.8, which summarizes the median execution time ratio, as well as the proportion of instances that either exceeded or remained below the baseline.

5.2.1 Execution Time and Efficiency Analysis

GP+EI

To complement the analysis of solution quality and variability, the computational efficiency of both AutoML methods was further assessed across problem sizes. To start with, GP+EI demonstrated progressive efficiency improvements as the problem size escalated. Out of 24 configurations, 15 were executed faster than the baseline (ratio < 1), with nine exceeding the baseline cost. Five out of these nine instances were part of the smallest problem size configuration (**25 $\times$ 5**). The median execution time ratio (1.42) revealed that an extra cost of 42% was observed for this configuration. All of the instances ranged between 0.82 (best case) and 1.91 (worst case). Provided this, the main inquiry now is to determine if this result is of concern.

Surveys noted that manual tuning often tests many hyperparameter settings before converging on a final choice. (Eiben & Smit 2011). Thus, there is **hidden implicit cost** behind the baseline approach. An overly modest yet critical assumption is that traditional tuning involves at least one additional trial. In such a case, total effort would double, making any ratio < 2 computationally superior. Given that none of the instances reached this threshold, their efficiency remains favorable even under conservative assumptions.

In practice, however, arriving at a final configuration often takes more than two intuitive attempts, especially when the same settings are expected to blanket across varying problem sizes. This necessitates additional fine-tuning readjustments, further amplifying the true computational cost. It would not be unreasonable to estimate the implicit effort at **five to ten times** that of the recorded runs. In fact, it could even be considered prudent. Consequently, this finding truly unshackles researchers from tedious and intuitively procedural tasks, allowing them to use their precious working hours to optimize other areas of the algorithm instead. Contemplating this, the previous inquiry now shifts, less to whether the result is of concern, and more to recognizing how compelling the alternative truly is.

The parenthesis on the implicit cost probe, although seemingly gripping, is not the only intriguing guide thread. As we expand the analysis to the rest of the problem sizes, GP+EI gradually conveys better computational efficiency. For **50×5**, with values ranging from 0.64 to 1.15, the median ratio converged to 0.96, representing a 4% cost reduction. Three instances had ratios < 1 and three > 1 . For **50×10**, all six instances had ratios < 1 , ranging from 0.72 to 0.87, with a median 23% improvement. At last, **100×10**, displayed cutbacks in five out of six instances, having 13% less median expense.

The pattern starts to become apparent, especially in view of Table 5.8. As problem sizes increase, GP+EI exhibits a sturdier and more consistent reduction in explicit cost as well. The two largest configurations examined, included only one instance without a recorded benefit. All of this contributes to GP+EI’s profile as a promising option. A broader reflection that outlines the general benefit follows at the end of the section, after briefly shifting focus to TPE.

TPE

TPE also showed favorable computational performance, with 18 out of 24 instances converging to the best configuration faster than the baseline. When replicating the analysis by problem size, akin results to the ones from GP+EI emerge.

In detail, by examining Table 5.8, it is observed that for **25×5**, four instances fell below baseline cost, but two exhibited notable overheads, particularly one outlier at 2.34, resulting in a median of 0.81. In contrast, **50×5** presented a reversed pattern: only two instances ran cheaper than the baseline, with a median of 1.04. However, in similar fashion to GP+EI, performance improved consistently with scale. For **50×10**, all instances showed gains, with a tightly clustered range and a median of 0.78. The trend continued in **100×10**, where all six instances again ran faster than baseline, though with slightly higher spread, yielding a median of 0.92.

The occasional inefficiency in mid sized problems can be again mitigated by invoking the notion of implicit cost discussed in the GP+EI segment above, which remains equally applicable here. With this in mind, TPE exhibited performance comparable to, and perhaps slightly better than, GP+EI (though such advantages appear marginal).

Table 5.8: Execution time summary by problem size for both methods

		GP+EI				TPE			
Size	Instances	>1	<1	Median	Ratio	Δ%	>1	<1	Median
25×5	6	5	1	1.42		+42%	2	4	0.81
50×5	6	3	3	0.96		-4%	4	2	0.87
50×10	6	0	6	0.77		-23%	0	6	1.01
100×10	6	1	5	0.87		-13%	0	6	0.80

Conclusive Overlying Trend

Both TPE and GP+EI show consistent improvement in execution time as problem size increases, indicating strong scalability and cost-efficiency in larger, more complex instances. While smaller configurations occasionally incur overhead, these are not only counterbalanced but also overshadowed by the implicit costs of the baseline method. This pattern complements the earlier quality results and will be revisited in the discussion.

5.3 Configurations Tested Deltas

In addition to improvements in solution quality and runtime efficiency, it is worth noting that these results were achieved while examining substantially fewer configurations than the manually tuned baseline. All 48 configuration deviation values recorded were non-positive, with 46 strictly negative, indicating that the AutoML methods consistently required fewer evaluations. Notably, 44 of these fell within the range of -5 to -10 , suggesting a meaningful reduction in search effort. Despite this lighter traversal, the solution ratios reported across instances remained higher than the baseline in nearly all cases, implying that search assisted by surrogates explored the hyperparameter space more effectively, even under constrained evaluation budgets.

Furthermore, the computational efficiency discussed in Section 5.2 opens the possibility of extending early stopping and time limit thresholds without breaching parity with the baseline’s resource footprint. This allowance could, in turn, lead to further refinement in solution quality, particularly in more complex instance settings where surrogate models tend to shine. Together, these findings reinforce the practicality and adaptability of AutoML tuning strategies under real world constraints.

5.4 Discussion

The results presented in this chapter converge toward a coherent narrative: surrogate-based AutoML methods, specifically GP+EI and TPE, can provide measurable improvements in both solution quality and computational efficiency when applied to the hyperparameter tuning of metaheuristics. However, these gains are neither uniformly distributed nor equally reliable across all problem scales and structural configurations. This discussion synthesizes those findings to outline the broader implications, caveats, and avenues for future exploration.

5.4.1 General Observations

Both methods consistently outperformed the manually tuned baseline in terms of weighted tardiness, with 45 out of 48 configurations yielding improved objective values. The other three valuations showed increases of only +1.38%, +0.06%, and +0.85%, relatively minimal, considering these represent the worst cases observed.

GP+EI demonstrated more stable improvements, particularly in larger problem sizes and instances with higher setup intensity (α). TPE, while more volatile and generally less optimal, still delivered hefty improvements, though without clearly responding to the structural control parameters of the generated instances.

In terms of computational cost, both methods outperformed the baseline in 33 out of 48 instances. The remaining 15 instances, all with execution time ratios within the range (1, 2), would still be considered less expensive when accounting for implicit cost. Notably, cost reductions became more substantial as problem complexity increased. This not only highlights the scalability of search with surrogates, but also reinforces its suitability for large scale scheduling domains.

Finally, the Solution Ratios demonstrated stronger exploration of the search space, as expected, despite fewer configurations being tested. This is significant, as it suggests improved performance and adaptability achieved with limited trials and reduced run-time. If this saved budget were reallocated to extend the computational allowance of AutoML methods, the potential for further improvement could be substantial.

5.4.2 Uncovering the Best-Case Scenarios

The most decisive benefits were observed in high-complexity scenarios. When $\alpha=2$ (**heavy setup intensity**) and $\tau=0.8$ (**tightened deadlines**), both methods achieved their best solution quality outcomes, particularly in the 100×10 configuration. However, the results suggest that the methods were more sensitive to setup intensity than to deadline tightness. Additionally, these configurations consistently showed execution time ratios below 1, reinforcing that the best-performing solutions did not incur a computational premium. Overall, this pattern aligns more strongly with GP+EI than with TPE, indicating that the “best-case scenario” is best represented by GP+EI under these structural conditions.

Another area of clarity lies in the 50×10 and 100×10 cases, where both methods exhibited convergence toward better optima, more compact RSI values, and fewer outliers. These results suggest that instance dimensionality, when paired with adequate structural constraints, enhances the optimizer’s ability to generalize.

Despite promising results, there are nuances that warrant deeper scrutiny. For GP+EI, the 50×5 configuration revealed unexplained regressions and brittle performance in select instances, indicating that the acquisition-exploitation balance may need adaptive control. TPE, though effective at scale, exhibited sporadic instability in mid sized problems, likely due to its reliance on nonparametric density estimation under low sampling regimes. Kernel bandwidth tuning or hybrid acquisition heuristics may alleviate this sensitivity.

Furthermore, the stochastic nature of the GA means that even averaged outcomes can mask variability, particularly in configurations close to performance thresholds. Incorporating repeated tuning trials or formal sensitivity analyses would provide a more robust performance characterization.

In short, the top three best-case scenarios, based on a combination of low error rates, stable RSI, and low computational cost are displayed in Table 5.9.

Rank	Method	α	τ	Size
1 st	GP+EI	2	0.8	100×10
2 nd	GP+EI	2	0.5	100×10
3 rd	TPE	2	0.8	100×10

Table 5.9: Top-Ranked Configurations Based on Performance, Stability, and Computational Efficiency

Concluding Insight

Collectively, the findings reaffirm the value of AutoML in metaheuristic tuning, specially in high-complexity, large scale instances where both performance gains and cost savings are most pronounced. While GP+EI stands out for its balance of stability and performance, TPE remains a viable alternative that operates effectively under the right conditions. The path forward involves enhancing reliability in volatile regions and refining integration strategies for each method. These considerations form the basis for the concluding reflections and recommendations outlined in the next chapter.

Chapter 6 Conclusion

6.1 Summary of Findings

This thesis evaluated the viability and practical utility of Automated Machine Learning (AutoML) techniques, particularly Bayesian Optimization methods, in the context of hyperparameter tuning for metaheuristics. A Genetic Algorithm (GA) was used to solve a complex scheduling problem focused on minimizing weighted tardiness in unrelated parallel machine environments with sequence-dependent setup times. Two AutoML strategies, Gaussian Process with Expected Improvement (GP+EI) and Tree-structured Parzen Estimators (TPE), were benchmarked against a manually tuned baseline across varying problem scales and structural configurations.

The findings revealed that both methods frequently matched or surpassed the manually tuned baseline, with the most significant improvements observed in large scale instances characterized by high setup intensity and tightened temporal constraints. GP+EI consistently outperformed in terms of stability and responsiveness, whereas TPE, though sometimes achieving strong results, exhibited greater variance.

Beyond solution quality, a notable contribution of this work is the demonstration of computational efficiency achieved through AutoML techniques using surrogates. These methods not only reduced the explicit number of evaluations required but also implicitly minimized the human effort involved in trial-and-error tuning. Particularly in high-dimensional or computationally demanding tasks, AutoML emerged as a viable and scalable alternative to traditional tuning practices.

Another key contribution is its interdisciplinary focus. By adapting AutoML frameworks, originally conceived for model selection and tuning in machine learning, to classical optimization problems in operations research, the study illustrates how automation can bridge methodological divides. This integration enables the adoption of native ML tools in traditional optimization workflows, offering a platform for further convergence of the two fields.

The conclusions drawn herein are framed within several constraints. The study focused exclusively on a single metaheuristic (GA) and a fixed benchmark dataset of synthetic instances. Additionally, while stochastic variability was mitigated via repeated runs, it remains a source of uncertainty, particularly in mid scale problem instances. The AutoML frameworks used also imposed some configurational limitations, particularly regarding surrogate customization and acquisition strategies.

6.2 Future Work

This study opens multiple promising avenues for continued research at the intersection of AutoML and operations research. One immediate direction involves extending the experimental scope beyond Genetic Algorithms to include other metaheuristic paradigms such as Ant Colony Optimization, Simulated Annealing, Tabu Search, or hybrid evolutionary frameworks. Such diversification would test the generalizability of AutoML’s benefits across algorithmic structures with different stochastic dynamics and parameter topologies.

Another key extension involves the adoption of real-world industrial datasets in addition to the synthetic benchmark instances used here. While the dataset generation followed established procedures and preserved complexity, incorporating industrial case studies, e.g., from manufacturing, logistics, or energy systems, would improve the external validity and practical transferability of findings.

From a methodological perspective, several enhancements could improve the reliability and performance of AutoML tuning in metaheuristic settings. First, expanding the statistical treatment of stochasticity, through repeated AutoML runs, ensemble-based acquisition strategies, or variance-aware optimization, could help stabilize outcomes under noise-heavy conditions. Second, incorporating multi-fidelity modeling or adaptive evaluation strategies may reduce computational cost, especially in early-stage exploration. Techniques such as Successive Halving, Hyperband, or dynamic fidelity scaling could support more sample-efficient searches.

Furthermore, sensitivity analyses around search space design and parameter importance could provide deeper insights into which hyperparameters most significantly influence performance, thus informing both modelers and future automation layers. This aligns with the thesis’s observation that fixed-range definitions, while practical, may still introduce configuration bias.

While this thesis focused on the practical application of AutoML in tuning meta-

heuristics, future work could also explore theoretical modeling of the interaction between surrogate model behavior and the underlying landscape of combinatorial problems. Understanding when, how, and why surrogate models succeed or fail in these settings may unlock more principled strategies for integrating ML-based optimizers with classical search techniques.

Finally, a practical next step would be to extend the current AutoML wrapper into a modular benchmarking suite for metaheuristic tuning. This tool could support plug-and-play integration of different algorithms (e.g., PSO, ACO), benchmark instances, and HPO strategies with minimal reconfiguration. By open sourcing such a framework, future research could move toward standardized comparisons across methods, while also facilitating reproducibility and industrial experimentation.

In sum, the research landscape at the intersection of AutoML and operations research remains vast, unevenly charted, and rich with untapped opportunity. While this thesis has lit a modest torch in one corridor of that space, countless other corridors remain unexplored, each with their own peculiar constraints, surrogate behaviors, and algorithmic intricacies. With deeper theoretical modeling, sharper statistical tooling, and a broader algorithmic lens, future work can illuminate not just how AutoML performs optimization, but why, and in doing so, help transform this methodological bridge into a thriving two-way avenue.

6.3 Closing Remarks

This work advocates for the thoughtful incorporation of AutoML techniques into operations research workflows. Under the right conditions and with informed configuration, these methods show clear promise not only in improving solution quality and reducing cost but also in reshaping how algorithmic tuning is approached in complex optimization tasks. While not a wholesale replacement for domain expertise, AutoML provides a flexible and evolving complement, one that merits deeper exploration across the optimization community.

Bibliography

- [1] Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp.2623–2631.
- [2] Avgerinos, I., Mourtos, I., Vatikiotis, S. and Zois, G., 2023. Weighted tardiness minimization for unrelated machines with sequence-dependent and resource-constrained setups. *International Journal of Production Research*, 62(1–2), pp.359–379. Available at: <https://doi.org/10.1080/00207543.2023.2275634>.
- [3] Awad, M., Ismail, A. and Aljarah, I., 2020. AutoML-based parameter tuning for Ant Colony Optimization. *Expert Systems with Applications*, 160, p.113707.
- [4] Balandat, M., Karrer, B., Jiang, D.R., Daulton, S., Letham, B., Wilson, A.G. and Bakshy, E., 2020. BoTorch: A framework for efficient Monte-Carlo Bayesian optimization. In: *Advances in Neural Information Processing Systems*, 33, pp.21524–21538.
- [5] Bergstra, J. and Bengio, Y., 2012. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(Feb), pp.281–305.
- [6] Bergstra, J., Yamins, D. and Cox, D.D., 2011. Hyperopt: A Python library for optimizing the hyperparameters of machine learning algorithms. In: *Proceedings of the 12th Python in Science Conference (SciPy)*, pp.13–20.
- [7] Birattari, M., Yuan, Z., Balaprakash, P. and Stützle, T., 2009. F-Race and iterated F-Race: An overview. In: T. Bartz-Beielstein et al., eds., *Experimental Methods for the Analysis of Optimization Algorithms*, Berlin: Springer, pp.311–336.

- [8] Bischl, B., Binder, M., Lang, M., Pfisterer, F., Rahnenführer, J. and Thomas, J., 2021. Hyperparameter optimization: Foundations, algorithms, best practices and open challenges. *arXiv preprint arXiv:2107.05847*.
- [9] Blum, C. and Roli, A., 2003. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys (CSUR)*, 35(3), pp.268–308.
- [10] Brochu, E., Cora, V.M. and de Freitas, N., 2010. A tutorial on Bayesian optimization of expensive cost functions. *arXiv preprint arXiv:1012.2599*.
- [11] Brucker, P., 2007. *Scheduling Algorithms*. 5th ed. Berlin: Springer.
- [12] Bubeck, S., Munos, R., Stoltz, G. and Szepesvári, C., 2011. X-armed bandits. *Journal of Machine Learning Research*, 12, pp.1655–1695.
- [13] Calandra, R., Seyfarth, A., Peters, J. and Deisenroth, M.P., 2014. Bayesian optimization for learning gaits under uncertainty. *Annals of Mathematics and Artificial Intelligence*, 76(1–2), pp.5–23.
- [14] De Jong, K.A., 2006. *Evolutionary Computation: A Unified Approach*. Cambridge, MA: MIT Press.
- [15] Eiben, A.E. and Smit, S.K., 2011. Parameter tuning for configuring and analyzing evolutionary algorithms. *Swarm and Evolutionary Computation*, 1(1), pp.19–31.
- [16] Ezugwu, A.E., Greeff, J. and Ho, Y.-S., 2023. A comprehensive study of groundbreaking machine learning research. *arXiv preprint arXiv:2308.00855*. Available at: <https://arxiv.org/abs/2308.00855>.
- [17] Falkner, S., Klein, A. and Hutter, F., 2018. BOHB: Robust and efficient hyperparameter optimization at scale. In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp.1437–1446.
- [18] Feurer, M. and Hutter, F., 2019. Hyperparameter optimization. In: F. Hutter et al., eds., *Automated Machine Learning: Methods, Systems, Challenges*. Cham: Springer, pp.3–33.
- [19] Feurer, M. et al., 2015. Efficient and robust automated machine learning. In: *Advances in Neural Information Processing Systems*, 28, pp.2962–2970.

- [20] Franceschi, L. et al., 2022. On the intractability of hyperparameter optimization. *Journal of Machine Learning Research*, 23(179), pp.1–62.
- [21] Frazier, P.I., 2018. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811*. Available at: <https://arxiv.org/pdf/1807.02811>.
- [22] Frazier, P.I., Powell, W.B. and Dayanik, S., 2009. The Knowledge-Gradient Policy for Correlated Normal Beliefs. *INFORMS Journal on Computing*, 21(4), pp.599–613.
- [23] Glover, F. and Kochenberger, G.A., eds., 2003. *Handbook of Metaheuristics*. Boston: Springer.
- [24] Goldberg, D.E., 1989. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley.
- [25] Graham, R.L. et al., 1979. Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics*, 5, pp.287–326.
- [26] Haupt, R.L. and Haupt, S.E., 2004. *Practical Genetic Algorithms*. 2nd ed. Hoboken, NJ: Wiley.
- [27] Holland, J.H., 1975. *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press.
- [28] Hutter, F., Hoos, H.H. and Leyton-Brown, K., 2011. Sequential model-based optimization for general algorithm configuration. In: *Proceedings of LION 5*. Berlin: Springer, pp.507–523.
- [29] Hutter, F., Kotthoff, L. and Vanschoren, J., eds., 2019. *Automated Machine Learning: Methods, Systems, Challenges*. Cham: Springer. Available at: <https://doi.org/10.1007/978-3-030-05318-5>.
- [30] Jamieson, K. and Talwalkar, A., 2016. Non-stochastic best arm identification and hyperparameter optimization. In: *Proceedings of AISTATS*, pp.240–248.
- [31] Jones, D.R., Schonlau, M. and Welch, W.J., 1998. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4), pp.455–492.
- [32] Kianpour, K., Zhang, X., Lu, C. and Naderi, B., 2023. DOE-based enhanced genetic algorithm for unrelated parallel machine scheduling to

minimize earliness–tardiness costs. *Journal of Intelligent Manufacturing*.
<https://doi.org/10.1007/s10845-023-02164-y>

- [33] Li, L. et al., 2017. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185), pp.1–52.
- [34] Liu, H., Simonyan, A. and Yang, Y., 2020. DARTS+: Improved Differentiable Architecture Search with Early Stopping. *arXiv preprint arXiv:2008.11267*.
- [35] Lobo, F.G., Lima, C.F. and Michalewicz, Z., 2007. Parameter setting in evolutionary algorithms. In: F.G. Lobo et al., eds., *Parameter Setting in Evolutionary Algorithms*. Berlin: Springer, pp.1–15.
- [36] López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Stützle, T. and Birattari, M., 2016. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3, pp.43–58.
- [37] Mantovani, R.G., Horváth, T., Rossi, A.L.D. et al., 2016. Better Trees: An empirical study on hyperparameter tuning of classification decision tree induction algorithms. *arXiv preprint arXiv:1812.02207*.
- [38] Misir, M. and Sebag, M., 2013. Algorithm portfolios and scheduling policies for submodular optimization. In: *LION 7: Learning and Intelligent Optimization*, pp.247–258.
- [39] Mitchell, M., 1998. *An Introduction to Genetic Algorithms*. Cambridge, MA: MIT Press.
- [40] Olson, R.S., Bartels, C., Urbanowicz, R.J. and Moore, J.H., 2018. Evaluation of a tree-based pipeline optimization tool for automating data science. *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pp.485–492.
- [41] Pinedo, M.L., 2016. *Scheduling: Theory, Algorithms, and Systems*. 5th ed. New York: Springer.
- [42] Rasmussen, C.E. and Williams, C.K.I., 2006. **Gaussian Processes for Machine Learning**. Cambridge, MA: MIT Press.
- [43] Rüther, M. and Rieck, C., 2024. Automated configuration of a grouping genetic algorithm using Bayesian optimization. *Algorithms*, 17(1), p.14.

- [44] Shahriari, B. et al., 2016. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1), pp.148–175.
- [45] Smith, J.E., 2008. Coevolving memetic algorithms: A review and progress report. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, 38(1), pp.28–37.
- [46] Snoek, J., Larochelle, H. and Adams, R.P., 2012. Practical Bayesian optimization of machine learning algorithms. In: *Advances in Neural Information Processing Systems*, 25, pp.2951–2959.
- [47] Vatikiotis, S., 2025. *Scheduling variants in manufacturing and process industries*. Ph.D. thesis, Athens University of Economics and Business.
- [48] Watanabe, S. and Hutter, F., 2022. c-TPE: Tree-structured Parzen Estimator with Inequality Constraints for Expensive Hyperparameter Optimization. arXiv preprint arXiv:2211.14411.
- [49] Zhang, R. and Demir, E., 2019. A new multi-objective mathematical model for energy-efficient job shop scheduling problems. *Computers & Operations Research*, 104, pp.114–130. <https://doi.org/10.1016/j.cor.2018.12.016>
- [50] Zöller, M.-A. and Huber, M.F., 2019. Survey on automated machine learning. *Journal of Artificial Intelligence Research*, 70, pp.409–472.